ТОО ”Колледж Хекслет”

Лабораторная работа №9

Конфигурация и событийная модель в Node.js

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_1. Конфигурация и событийная модель в Node.js.

Условие: RO4_1/lesson9/Задание.md

Задания

Общее для всех вариантов условие:

1.Инициализировать проект через npm init.

2.Установить зависимости: dotenv (основная) и nodemon (для разработки).

3.Настроить в package.json скрипт dev, который отслеживает изменения файлов .js и .env.

4.Использовать EventEmitter для связи между модулями.

5.Вынести все критические параметры (лимиты, имена, порты) в файл .env.

Легенда: Вы разрабатываете модуль безопасности для банка, который отслеживает подозрительные переводы.

Техническое задание:

1. Файл .env: Создать переменную LIMIT, определяющую сумму, выше которой перевод считается подозрительным.

2. События: Реализовать два события:

transfer — срабатывает при обычном переводе.

fraud-alert — срабатывает, если сумма превышает LIMIT.

3. Логика:

Раз в 3 секунды генерировать случайную сумму перевода.

Если сумма выше лимита — «выбрасывать» событие тревоги и выводить в консоль сообщение: «[ALARM] Подозрительная активность: {сумма}!».

Если ниже — выводить: «Успешный перевод: {сумма}».

Ход выполнения

Лимит, интервал и максимальная сумма заданы в .env.example. dotenv читает .env относительно каталога урока. EventEmitter связывает генератор переводов и мониторинг: сумма выше LIMIT вызывает fraud-alert, на лимите и ниже — transfer. Интервал по умолчанию 3 секунды. Некорректный LIMIT

останавливает запуск с понятным сообщением.

Проект: RO4_1/lesson9

Из корня репозитория: npm ci. В каталоге урока: cp .env.example .env, затем npm start или npm run dev.

package.json

package.json

```
{
  "name": "lesson9-transaction-monitor",
  "version": "1.0.0",
  "description": "Система мониторинга банковских транзакций",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon --watch '*.js' --watch '.env' index.js"
  },
  "dependencies": {
    "dotenv": "^16.6.1"
  },
  "devDependencies": {
    "nodemon": "^3.1.10"
  }
}
```

Листинг 1. package.json

.env.example

.env.example

```
LIMIT=50000
INTERVAL_MS=3000
MAX_AMOUNT=100000
```

Листинг 2. .env.example

events.js

events.js

```
const EventEmitter = require('events');

const events = new EventEmitter();

module.exports = events;
```

Листинг 3. events.js

monitor.js

```
monitor.js
```

```
const events = require('./events');

function checkTransfer(sum, limit) {
    if (sum > limit) {
        events.emit('fraud-alert', sum);
    } else {
        events.emit('transfer', sum);
    }
}

module.exports = { checkTransfer };
```

Листинг 4. monitor.js

index.js

```
index.js
```

```
const path = require('path');
require('dotenv').config({ path: path.join(__dirname, '.env') });

const events = require('./events');
const { checkTransfer } = require('./monitor');

// Задание 1:
let limit = Number(process.env.LIMIT);
const intervalMs = Number(process.env.INTERVAL_MS || 3000);
const maxAmount = Number(process.env.MAX_AMOUNT || 100000);
if (!Number.isFinite(limit) || limit <= 0 || !Number.isSafeInteger(intervalMs) ||
intervalMs <= 0 || !Number.isSafeInteger(maxAmount) || maxAmount <= 0) {
    console.error('Задайте положительный LIMIT и корректные INTERVAL_MS/MAX_AMOUNT в
.env (образец: .env.example)');
    process.exit(1);
}

console.log(`Лимит перевода: ${limit}`);

// Задание 2:
events.on('transfer', (amount) => {
    console.log(`Успешный перевод: ${amount}`);
});

events.on('fraud-alert', (amount) => {
    console.log(`[ALARM] Подозрительная активность: ${amount}!`);
});

// Задание 3:
function transfer() {
    let sum = Math.floor(Math.random() * maxAmount) + 1;
    checkTransfer(sum, limit);
}

console.log('Мониторинг переводов запущен');
```

Листинг 5. index.js (часть 1)

index.js

```
34 transfer();
35 setInterval(transfer, intervalMs);
```

Листинг 6. index.js (часть 2)

Проверка результата

Проверено: 05.11.2026 13:44 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Ровно на лимите → transfer

• Выше лимита → fraud-alert

• Отсутствующий/некорректный LIMIT останавливает запуск с понятной ошибкой

• .env читается относительно каталога урока, параметры вынесены в .env.example

Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.